Credit Card Fraud Detection Using Deep Learning

This notebook implements and compares four deep learning architectures: MLP, 1D CNN, LSTM and Autoencoder. It evaluates BCE, class-weighted BCE and Focal Loss for MLP/CNN, and adds an LSTM trained with standard BCE as an additional architecture comparison. It also performs validation-based threshold selection, test evaluation, visualizations, and optional SHAP/LIME explanations.

Important methodological note about LSTM: The supplied dataset does not contain customer/card identifiers needed to build genuine per-customer transaction histories. Therefore, this notebook uses the 30 input features as a feature sequence (30 steps, one value per step). This is an exploratory LSTM architecture comparison, not a temporal customer-transaction model. V1–V28 are anonymized PCA components, so their ordering has no established temporal or spatial meaning. Interpret the LSTM results with this limitation in mind.

Run cells from top to bottom in Google Colab and upload the actual `creditcard.csv` file when prompted.

1. Install packages

```
!pip -q install shap lime
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for lime (pyproject.toml) ... done
```

2. Imports and reproducibility

```
import os, random, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn a
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (precision_score, recall_score, f1_score,
    average_precision_score, roc_auc_score, confusion_matrix,
    classification_report, precision_recall_curve)

SEED=42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
```

```
print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow: 2.21.0
GPU: []
```

3. Upload and load dataset

```
from google.colab import files
import pandas as pd

# Upload the actual creditcard.csv file (not archive.zip)
uploaded = files.upload()

csv_files = [name for name in uploaded if name.lower().endswith('.csv
if not csv_files:
    raise ValueError('Please upload creditcard.csv as a .csv file, no

# Prefer the expected dataset filename if more than one CSV is upload
csv_name = next((name for name in csv_files if 'creditcard' in name.l
df = pd.read_csv(csv_name)

required_columns = {'Class', 'Time', 'Amount'}
missing_columns = required_columns - set(df.columns)
if missing_columns:
    raise ValueError(f'This does not look like the expected credit ca

print('Dataset loaded successfully!')
print('File:', csv_name)
print('Dataset shape:', df.shape)
display(df.head())
```

Choose files creditcard.csv

creditcard.csv(text/csv) - 150828752 bytes, last modified: 20/09/2019 - 100% done

```
Saving creditcard.csv to creditcard.csv
Dataset loaded successfully!
File: creditcard.csv
Dataset shape: (284807, 31)
```

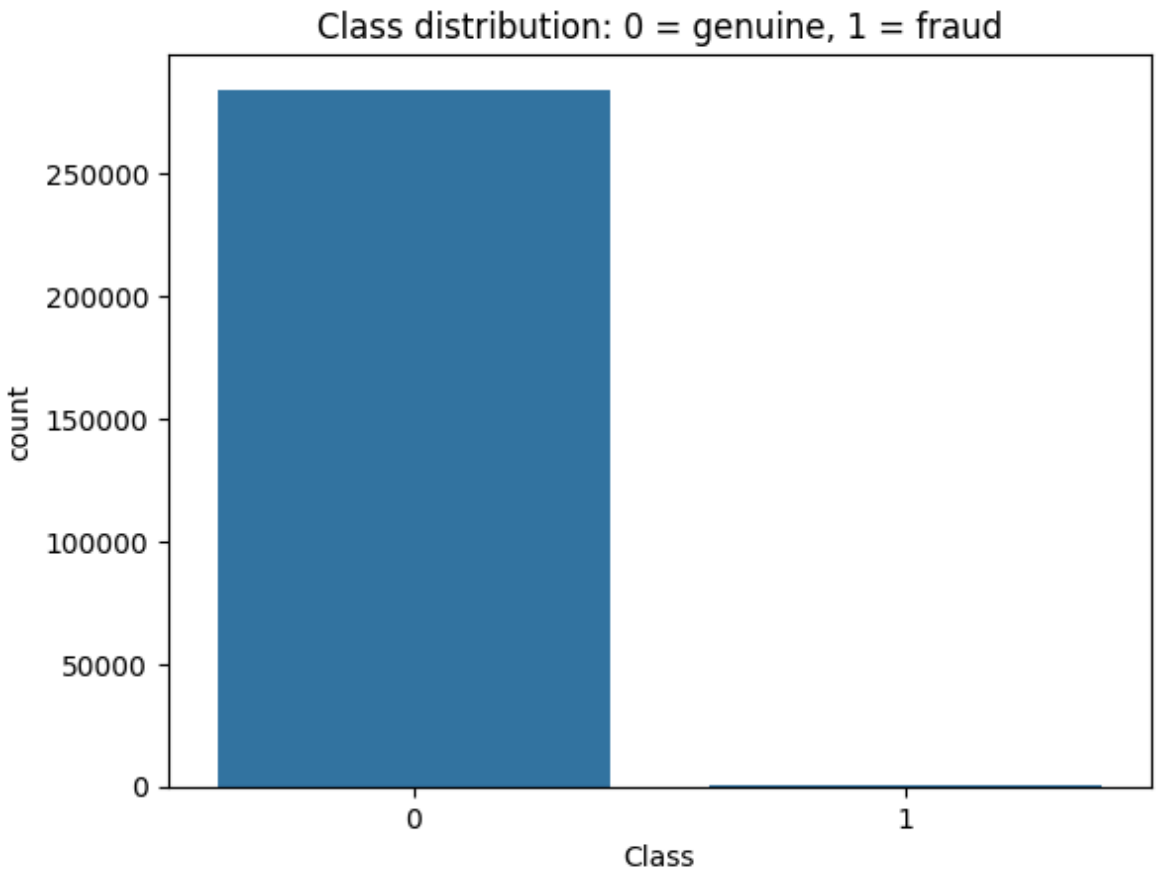
	Time	V1	V2	V3	V4	V5	V6	V
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.23959
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.07880
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.79146
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.23760
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.59294

5 rows × 31 columns

4. Inspect dataset and class imbalance

```
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
print(df["Class"].value_counts())
print((df["Class"].value_counts(normalize=True)*100).round(4))
sns.countplot(data=df, x="Class")
plt.title("Class distribution: 0 = genuine, 1 = fraud")
plt.show()
assert "Class" in df.columns and df.isnull().sum().sum()==0
```

```
Missing values: 0
Duplicate rows: 1081
Class
0    284315
1       492
Name: count, dtype: int64
Class
0    99.8273
1     0.1727
Name: proportion, dtype: float64
```



5. Split into training, validation and test sets

```
X=df.drop(columns=["Class"]).copy()
y=df["Class"].astype("int32").copy()
X_train, X_temp, y_train, y_temp=train_test_split(
```

```
    X,y,test_size=0.40,stratify=y,random_state=SEED)
X_val, X_test, y_val, y_test=train_test_split(
    X_temp,y_temp,test_size=0.50,stratify=y_temp,random_state=SEED)
print("Train",X_train.shape,"fraud",int(y_train.sum()))
print("Validation",X_val.shape,"fraud",int(y_val.sum()))
print("Test",X_test.shape,"fraud",int(y_test.sum()))
```

```
Train (170884, 30) fraud 295
Validation (56961, 30) fraud 98
Test (56962, 30) fraud 99
```

6. Scale Time and Amount using training data only

```
feature_names=X_train.columns.tolist()
scaler=StandardScaler()
scale_cols=["Time","Amount"]
X_train.loc[:,scale_cols]=scaler.fit_transform(X_train[scale_cols])
X_val.loc[:,scale_cols]=scaler.transform(X_val[scale_cols])
X_test.loc[:,scale_cols]=scaler.transform(X_test[scale_cols])
X_train_np=X_train.to_numpy(dtype=np.float32)
X_val_np=X_val.to_numpy(dtype=np.float32)
X_test_np=X_test.to_numpy(dtype=np.float32)
y_train_np=y_train.to_numpy(dtype=np.float32)
y_val_np=y_val.to_numpy(dtype=np.int32)
y_test_np=y_test.to_numpy(dtype=np.int32)
print("Input feature count:",X_train_np.shape[1])
```

```
Input feature count: 30
```

7. Calculate class weights

```
w=compute_class_weight(class_weight="balanced",classes=np.array([0,1]
class_weights={0:float(w[0]),1:float(w[1])}
print(class_weights)
```

```
{0: 0.5008646512963908, 1: 289.63389830508476}
```

8. Define loss functions and deep learning architectures

```
def focal_loss(gamma=2.0, alpha=0.75):
    def loss(y_true,y_pred):
        yt=tf.cast(tf.reshape(y_true,(-1,1)),tf.float32)
        yp=tf.clip_by_value(tf.cast(y_pred,tf.float32),1e-7,1-1e-7)
        pt=yt*yp+(1-yt)*(1-yp)
        at=yt*alpha+(1-yt)*(1-alpha)
        return tf.reduce_mean(-at*tf.pow(1-pt,gamma)*tf.math.log(pt))
    return loss
```

```
def build_mlp(n):
    return tf.keras.Sequential([
        tf.keras.layers.Input((n,)),
        tf.keras.layers.Dense(64,activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(32,activation="relu"),
        tf.keras.layers.Dropout(.20),
        tf.keras.layers.Dense(1,activation="sigmoid")])

def build_cnn(shape):
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape),
        tf.keras.layers.Conv1D(64,3,padding="same",activation="relu")
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Conv1D(32,3,padding="same",activation="relu")
        tf.keras.layers.GlobalMaxPooling1D(),
        tf.keras.layers.Dense(32,activation="relu"),
        tf.keras.layers.Dropout(.30),
        tf.keras.layers.Dense(1,activation="sigmoid")])



def build_lstm(shape):
    # Treats each input feature as one step in a feature sequence.
    # This is not a customer-level transaction-time sequence.
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape),
        tf.keras.layers.LSTM(32, dropout=0.20),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dropout(0.20),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

def compile_model(model,loss="binary_crossentropy"):
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),loss=loss)
    return model

def get_callbacks():
    return [tf.keras.callbacks.EarlyStopping(monitor="val_loss",patie

EPOCHS=15
BATCH_SIZE=512
```

9. Train MLP with standard Binary Cross-Entropy (baseline)

```
mlp_bce=compile_model(build_mlp(X_train_np.shape[1]))
hist_mlp_bce=mlp_bce.fit(X_train_np,y_train_np,validation_data=(X_val
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),ver
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 0.0582 - val_loss: 0.
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0051 - val_loss: 0.
```

```
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0042 - val_loss: 0.
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0039 - val_loss: 0.
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0036 - val_loss: 0.
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0033 - val_loss: 0.
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0030 - val_loss: 0.
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0028 - val_loss: 0.
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0029 - val_loss: 0.
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0029 - val_loss: 0.
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0027 - val_loss: 0.
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0027 - val_loss: 0.
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0025 - val_loss: 0.
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.0023 - val_loss: 0.
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.0023 - val_loss: 0.
```

10. Train MLP with class-weighted BCE

```
mlp_weighted=compile_model(build_mlp(X_train_np.shape[1]))
hist_mlp_weighted=mlp_weighted.fit(X_train_np,y_train_np,validation_d
    epochs=EPOCHS,batch_size=BATCH_SIZE,class_weight=class_weights,
    callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - loss: 0.2938 - val_loss: 0.
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1579 - val_loss: 0.
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1503 - val_loss: 0.
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1206 - val_loss: 0.
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1229 - val_loss: 0.
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1134 - val_loss: 0.
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - loss: 0.1002 - val_loss: 0.
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1104 - val_loss: 0.
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.1119 - val_loss: 0.
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.0921 - val_loss: 0.
```

11. Train MLP with Focal Loss

```
mlp_focal=compile_model(build_mlp(X_train_np.shape[1]),focal_loss())
hist_mlp_focal=mlp_focal.fit(X_train_np,y_train_np,validation_data=(X
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),ver
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 0.0031 - val_loss: 0.
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 9.7598e-04 - val_loss
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 8.0475e-04 - val_loss
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 6.8384e-04 - val_loss
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 6.1602e-04 - val_loss
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 6.0302e-04 - val_loss
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 5.2429e-04 - val_loss
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 5.1127e-04 - val_loss
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 5.0453e-04 - val_loss
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 5.0404e-04 - val_loss
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 4.5669e-04 - val_loss
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 4.2796e-04 - val_loss
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 3.7234e-04 - val_loss
```

12. Prepare CNN input and train CNN with BCE

```
X_train_cnn=X_train_np.reshape(-1,X_train_np.shape[1],1)
X_val_cnn=X_val_np.reshape(-1,X_val_np.shape[1],1)
X_test_cnn=X_test_np.reshape(-1,X_test_np.shape[1],1)
cnn_bce=compile_model(build_cnn((X_train_cnn.shape[1],1)))
hist_cnn_bce=cnn_bce.fit(X_train_cnn,y_train_np,validation_data=(X_va
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),ver
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 14s 36ms/step - loss: 0.0382 - val_loss:
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 37ms/step - loss: 0.0072 - val_loss:
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 0.0050 - val_loss:
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 0.0042 - val_loss:
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0040 - val_loss:
Epoch 6/15
```

```
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 37ms/step - loss: 0.0037 - val_loss:
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 39ms/step - loss: 0.0037 - val_loss:
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0036 - val_loss:
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 0.0035 - val_loss:
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0034 - val_loss:
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0033 - val_loss:
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 32ms/step - loss: 0.0032 - val_loss:
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.0032 - val_loss:
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0031 - val_loss:
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 0.0030 - val_loss:
```

13. Train CNN with class-weighted BCE

```
cnn_weighted=compile_model(build_cnn((X_train_cnn.shape[1],1)))
hist_cnn_weighted=cnn_weighted.fit(X_train_cnn,y_train_np,validation_
    epochs=EPOCHS,batch_size=BATCH_SIZE,class_weight=class_weights,
    callbacks=get_callbacks(),verbose=1)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 14s 37ms/step - loss: 0.4084 - val_loss:
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 0.2610 - val_loss:
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - loss: 0.2223 - val_loss:
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 34ms/step - loss: 0.2168 - val_loss:
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 35ms/step - loss: 0.1854 - val_loss:
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 13s 37ms/step - loss: 0.1803 - val_loss:
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 32ms/step - loss: 0.1808 - val_loss:
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - loss: 0.1669 - val_loss:
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 0.1621 - val_loss:
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 0.1574 - val_loss:
```

14. Train CNN with Focal Loss

```
cnn_focal=compile_model(build_cnn((X_train_cnn.shape[1],1)),focal_los
hist_cnn_focal=cnn_focal.fit(X_train_cnn,y_train_np,validation_data=(
```

```
    epochs=EPOCHS,batch_size=BATCH_SIZE,callbacks=get_callbacks(),ver
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 14s 37ms/step - loss: 0.0042 - val_loss:
Epoch 2/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 33ms/step - loss: 0.0011 - val_loss:
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 35ms/step - loss: 8.0861e-04 - val_lo
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 37ms/step - loss: 7.4259e-04 - val_lo
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - loss: 7.1106e-04 - val_lo
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 11s 34ms/step - loss: 7.1376e-04 - val_lo
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 7.0220e-04 - val_lo
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 20s 34ms/step - loss: 6.5758e-04 - val_lo
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - loss: 6.4536e-04 - val_lo
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 6.2592e-04 - val_lo
Epoch 11/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 5.9816e-04 - val_lo
Epoch 12/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 36ms/step - loss: 5.6594e-04 - val_lo
Epoch 13/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 19s 32ms/step - loss: 5.7003e-04 - val_lo
Epoch 14/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 21s 35ms/step - loss: 5.8392e-04 - val_lo
Epoch 15/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 12s 35ms/step - loss: 5.2382e-04 - val_lo
```

15. Train LSTM with standard Binary Cross-Entropy

The LSTM receives the same reshaped feature input as the CNN: `(30 features, 1 value per step)`. Since the dataset has no customer/card identifier, this is a feature-sequence experiment rather than a true temporal sequence of transactions. The results should be interpreted with that limitation.

```
lstm_bce = compile_model(build_lstm((X_train_cnn.shape[1], 1)))
hist_lstm_bce = lstm_bce.fit(
    X_train_cnn, y_train_np,
    validation_data=(X_val_cnn, y_val_np),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=get_callbacks(),
    verbose=1
)
```

```
Epoch 1/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 61ms/step - loss: 0.0625 - val_loss:
Epoch 2/15
```

```
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 67ms/step - loss: 0.0075 - val_loss:
Epoch 3/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 20s 60ms/step - loss: 0.0068 - val_loss:
Epoch 4/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 66ms/step - loss: 0.0066 - val_loss:
Epoch 5/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 20s 60ms/step - loss: 0.0069 - val_loss:
Epoch 6/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 65ms/step - loss: 0.0066 - val_loss:
Epoch 7/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 41s 66ms/step - loss: 0.0063 - val_loss:
Epoch 8/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 20s 60ms/step - loss: 0.0060 - val_loss:
Epoch 9/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 22s 67ms/step - loss: 0.0069 - val_loss:
Epoch 10/15
334/334 ━━━━━━━━━━━━━━━━━━━━ 41s 66ms/step - loss: 0.0059 - val_loss:
```

16. Train Autoencoder on genuine transactions

```
inp=tf.keras.layers.Input(shape=(X_train_np.shape[1],))
z=tf.keras.layers.Dense(16,activation="relu")(inp)
z=tf.keras.layers.Dense(8,activation="relu")(z)
z=tf.keras.layers.Dense(16,activation="relu")(z)
out=tf.keras.layers.Dense(X_train_np.shape[1],activation="linear")(z)
autoencoder=tf.keras.Model(inp,out)
autoencoder.compile(optimizer="adam",loss="mse")
X_train_genuine=X_train_np[y_train_np==0]
hist_ae=autoencoder.fit(X_train_genuine,X_train_genuine,
    validation_data=(X_val_np[y_val_np==0],X_val_np[y_val_np==0]),
    epochs=20,batch_size=BATCH_SIZE,callbacks=get_callbacks(),verbose=

def reconstruction_errors(model,data):
    rec=model.predict(data,batch_size=BATCH_SIZE,verbose=0)
    return np.mean(np.square(data-rec),axis=1)
ae_val_scores=reconstruction_errors(autoencoder,X_val_np)
ae_test_scores=reconstruction_errors(autoencoder,X_test_np)
```

```
Epoch 1/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 0.8558 - val_loss: 0.
Epoch 2/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.5469 - val_loss: 0.
Epoch 3/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4492 - val_loss: 0.
Epoch 4/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.4081 - val_loss: 0.
Epoch 5/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3832 - val_loss: 0.
Epoch 6/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3638 - val_loss: 0.
Epoch 7/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3506 - val_loss: 0.
Epoch 8/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3431 - val_loss: 0.
Epoch 9/20
```

```
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3385 - val_loss: 0.
Epoch 10/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3355 - val_loss: 0.
Epoch 11/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3333 - val_loss: 0.
Epoch 12/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3315 - val_loss: 0.
Epoch 13/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3300 - val_loss: 0.
Epoch 14/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3286 - val_loss: 0.
Epoch 15/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3274 - val_loss: 0.
Epoch 16/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - loss: 0.3263 - val_loss: 0.
Epoch 17/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.3253 - val_loss: 0.
Epoch 18/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3244 - val_loss: 0.
Epoch 19/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3236 - val_loss: 0.
Epoch 20/20
334/334 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3228 - val_loss: 0.
```

17. Optimize decision thresholds using validation predictions only

```
def choose_threshold(y_true,probs,objective="f1",min_precision=None):
    p,r,t=precision_recall_curve(y_true,probs)
    if len(t)==0: return 0.5
    p,r=p[:-1],r[:-1]
    f1=2*p*r/(p+r+1e-12)
    if min_precision is not None:
        eligible=np.where(p>=min_precision)[0]
        if len(eligible): return float(t[eligible[np.argmax(r[eligibl
    idx=int(np.argmax(r if objective=="recall" else f1))
    return float(t[idx])

models={
 "MLP - BCE":(mlp_bce,X_val_np,X_test_np),
 "MLP - Weighted BCE":(mlp_weighted,X_val_np,X_test_np),
 "MLP - Focal Loss":(mlp_focal,X_val_np,X_test_np),
 "CNN - BCE":(cnn_bce,X_val_cnn,X_test_cnn),
 "CNN - Weighted BCE":(cnn_weighted,X_val_cnn,X_test_cnn),
 "CNN - Focal Loss":(cnn_focal,X_val_cnn,X_test_cnn),
 "LSTM - BCE":(lstm_bce,X_val_cnn,X_test_cnn)
}
thresholds={}
for name,(model,xv,xt) in models.items():
    vp=model.predict(xv,batch_size=BATCH_SIZE,verbose=0).ravel()
    thresholds[name]=choose_threshold(y_val_np,vp)
thresholds["Autoencoder"]=choose_threshold(y_val_np,ae_val_scores)
print(thresholds)
```

```
{'MLP - BCE': 0.6594213843345642, 'MLP - Weighted BCE': 0.999252140522
```

18. Evaluate every model on the untouched test set

```
def evaluate(name,y_true,probs,threshold):
    pred=(probs>=threshold).astype(int)
    print("\\n",name,"threshold:",round(threshold,6))
    print(classification_report(y_true,pred,digits=4,zero_division=0)
    print("Confusion matrix:\\n",confusion_matrix(y_true,pred))
    return {"Model":name,"Threshold":threshold,
        "Precision":precision_score(y_true,pred,zero_division=0),
        "Recall":recall_score(y_true,pred,zero_division=0),
        "F1":f1_score(y_true,pred,zero_division=0),
        "PR-AUC":average_precision_score(y_true,probs),
        "ROC-AUC":roc_auc_score(y_true,probs)},pred

results=[]; test_probs={}; test_preds={}
for name,(model,xv,xt) in models.items():
    prob=model.predict(xt,batch_size=BATCH_SIZE,verbose=0).ravel()
    test_probs[name]=prob
    row,pred=evaluate(name,y_test_np,prob,thresholds[name])
    results.append(row); test_preds[name]=pred

row,pred=evaluate("Autoencoder",y_test_np,ae_test_scores,thresholds["
results.append(row); test_preds["Autoencoder"]=pred
```

```
results_df=pd.DataFrame(results).sort_values("PR-AUC",ascending=False
display(results_df)
```

Next steps: Generate code with `results_df` New interactive sheet

```
\n MLP - BCE threshold: 0.659421
              precision    recall  f1-score   support

           0     0.9997    0.9998    0.9998     56863
           1     0.8817    0.8283    0.8542        99

    accuracy                         0.9995     56962
   macro avg     0.9407    0.9140    0.9270     56962
weighted avg     0.9995    0.9995    0.9995     56962

Confusion matrix:\n [[56852    11]
 [   17    82]]
\n MLP - Weighted BCE threshold: 0.999252
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56863
           1     0.8750    0.7778    0.8235        99

    accuracy                         0.9994     56962
   macro avg     0.9373    0.8888    0.9116     56962
weighted avg     0.9994    0.9994    0.9994     56962

Confusion matrix:\n [[56852    11]
 [   22    77]]
\n MLP - Focal Loss threshold: 0.541768
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56863
           1     0.8778    0.7980    0.8360        99

    accuracy                         0.9995     56962
   macro avg     0.9387    0.8989    0.9179     56962
weighted avg     0.9994    0.9995    0.9994     56962

Confusion matrix:\n [[56852    11]
 [   20    79]]
\n CNN - BCE threshold: 0.383673
              precision    recall  f1-score   support

           0     0.9997    0.9997    0.9997     56863
           1     0.8438    0.8182    0.8308        99

    accuracy                         0.9994     56962
   macro avg     0.9217    0.9090    0.9152     56962
weighted avg     0.9994    0.9994    0.9994     56962

Confusion matrix:\n [[56848    15]
 [   18    81]]
\n CNN - Weighted BCE threshold: 0.994202
              precision    recall  f1-score   support

           0     0.9996    0.9994    0.9995     56863
           1     0.6909    0.7677    0.7273        99

    accuracy                         0.9990     56962
   macro avg     0.8453    0.8835    0.8634     56962
weighted avg     0.9991    0.9990    0.9990     56962
```

19. Plot confusion matrices and compare metrics

```
for name,pred in test_preds.items():
    plt.figure(figsize=(4,3))
```

```
    sns.heatmap(confusion_matrix(y_test_np,pred),annot=True,fmt="d",cm
    plt.title("Confusion Matrix: "+name); plt.xlabel("Predicted"); plt
    plt.show()

results_df.set_index("Model")[["Precision","Recall","F1","PR-AUC"]].p
    kind="bar",figsize=(12,5),ylim=(0,1),rot=35)
plt.title("Test-set comparison"); plt.ylabel("Score"); plt.tight_layou
```

```
weighted avg     0.9995    0.9995    0.9995     56962

Confusion matrix:\n [[56853    10]
 [   20    79]]
\n LSTM - BCE threshold: 0.095548
              precision    recall  f1-score   support

           0     0.9997    0.9998    0.9997     56863
           1     0.8526    0.8182    0.8351        99

    accuracy                         0.9994     56962
   macro avg     0.9262    0.9090    0.9174     56962
weighted avg     0.9994    0.9994    0.9994     56962

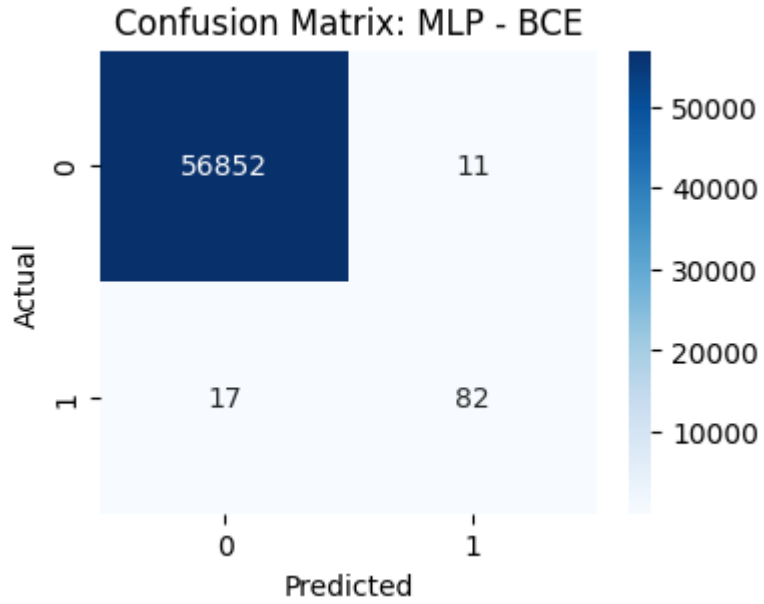
Confusion matrix:\n [[56849    14]
 [   18    81]]
\n Autoencoder threshold: 7.158791
              precision    recall  f1-score   support

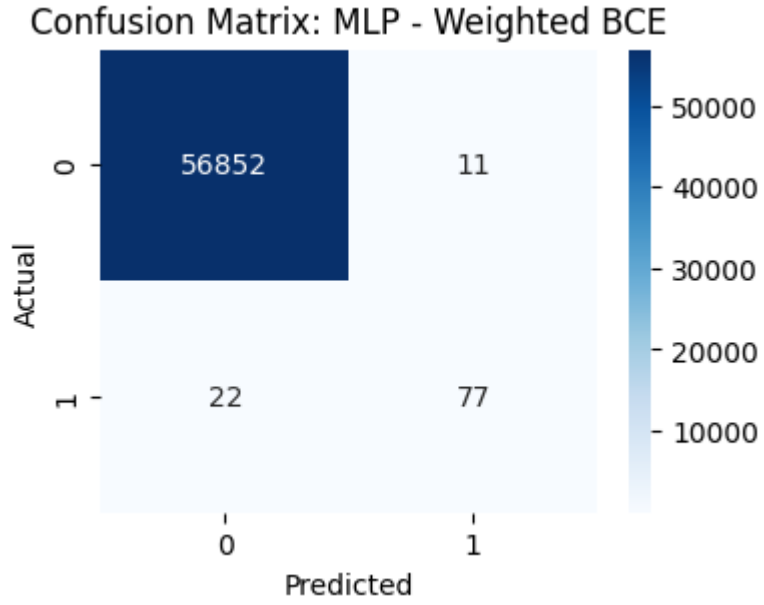
           0     0.9992    0.9985    0.9988     56863
           1     0.3830    0.5455    0.4500        99

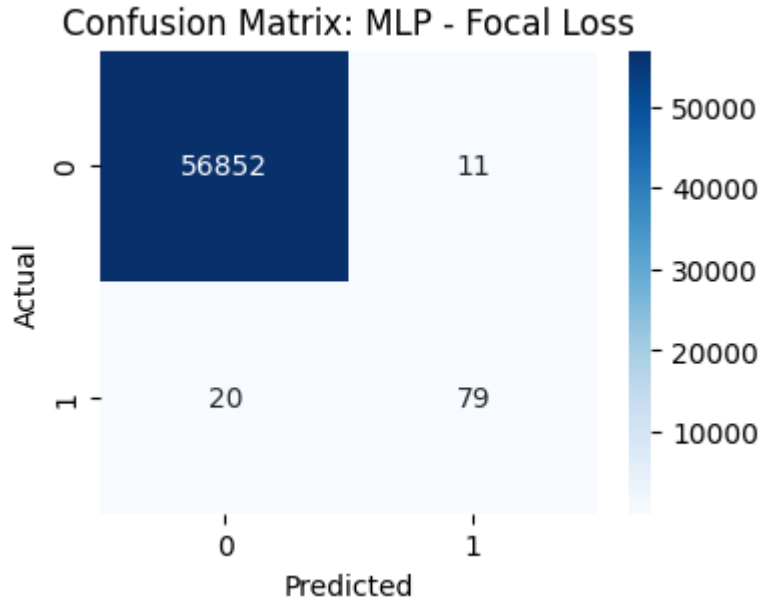
    accuracy                         0.9977     56962
   macro avg     0.6911    0.7720    0.7244     56962
weighted avg     0.9981    0.9977    0.9979     56962

Confusion matrix:\n [[56776    87]
 [   45    54]]
```

	Model	Threshold	Precision	Recall	F1	PR-AUC	ROC-AUC
0	MLP - BCE	0.659421	0.881720	0.828283	0.854167	0.843964	0.979354
2	MLP - Focal Loss	0.541768	0.877778	0.797980	0.835979	0.840167	0.978063
3	CNN - BCE	0.383673	0.843750	0.818182	0.830769	0.806705	0.968007
5	CNN - Focal Loss	0.514360	0.887640	0.797980	0.840426	0.806239	0.969684
6	LSTM - BCE	0.095548	0.852632	0.818182	0.835052	0.736545	0.928642
1	MLP - Weighted	0.999252	0.875000	0.777778	0.823529	0.726813	0.975387


Confusion Matrix: MLP - BCE
56852
11
17
82
Actual
Predicted
0
1
50000
40000
30000
20000
10000


Confusion Matrix: MLP - Weighted BCE
56852
11
22
77
Actual
Predicted
0
1
50000
40000
30000
20000
10000


Confusion Matrix: MLP - Focal Loss
56852
11
20
79
Actual
Predicted
0
1
50000
40000
30000
20000
10000


Confusion Matrix: CNN - BCE
56848
15
50000